Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

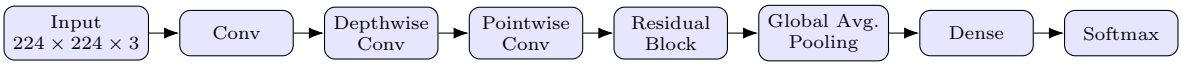
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

0.1 Effect of Weight Initialization

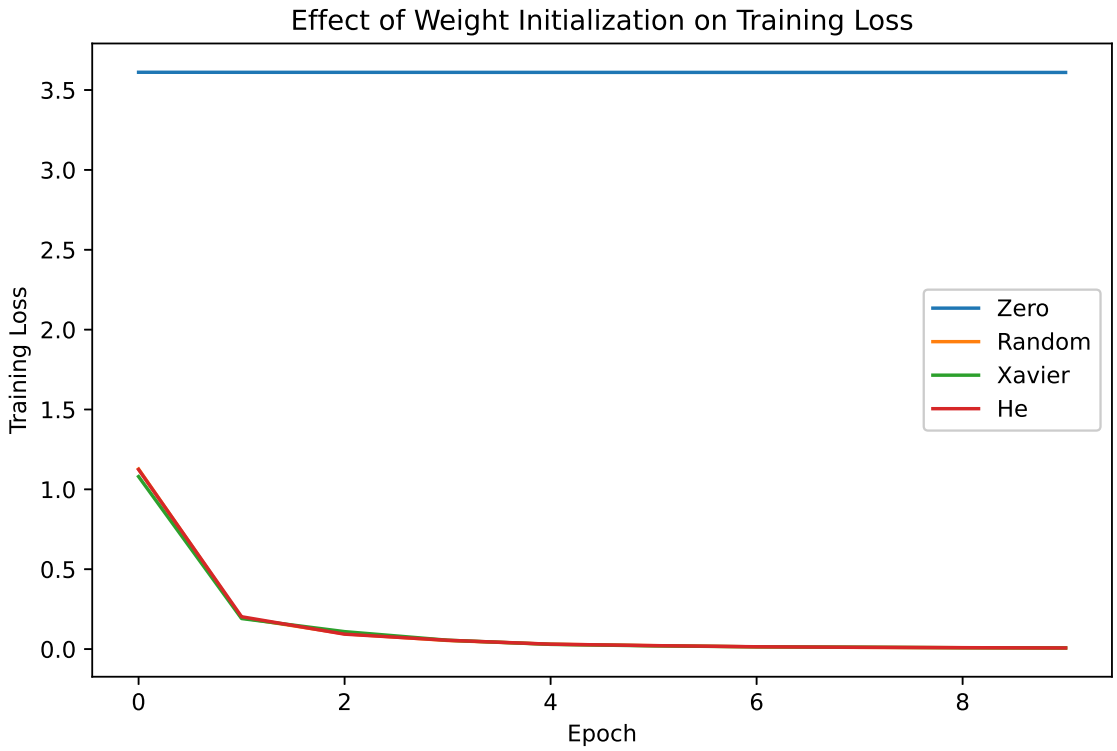


Figure 1: Training Loss vs. Epoch for Different Weight Initialization Methods

The plot compares the training loss curves obtained using different weight initialization methods.Generally the loss decreases with increasing epochs, with some initialization methods converging faster and more smoothly than others. This occurs because suitable initialization provides better starting weights and improves gradient propagation during training.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

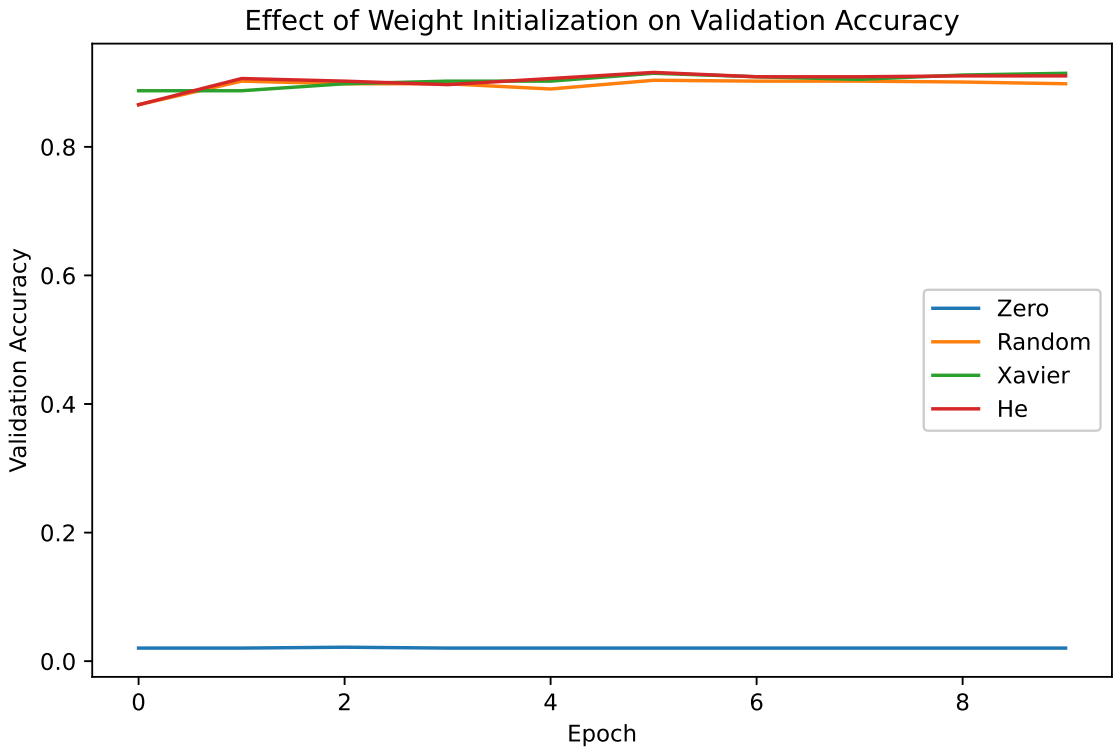


Figure 2: Validation Accuracy vs. Epoch for Different Weight Initialization Methods

The plot compares validation accuracy for the different weight initialization methods.Validation accuracy generally increases during training and approaches a stable value,some methods show faster or more stable convergence. This is because the initial weights influence optimization and the ability of the network to learn useful representations.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

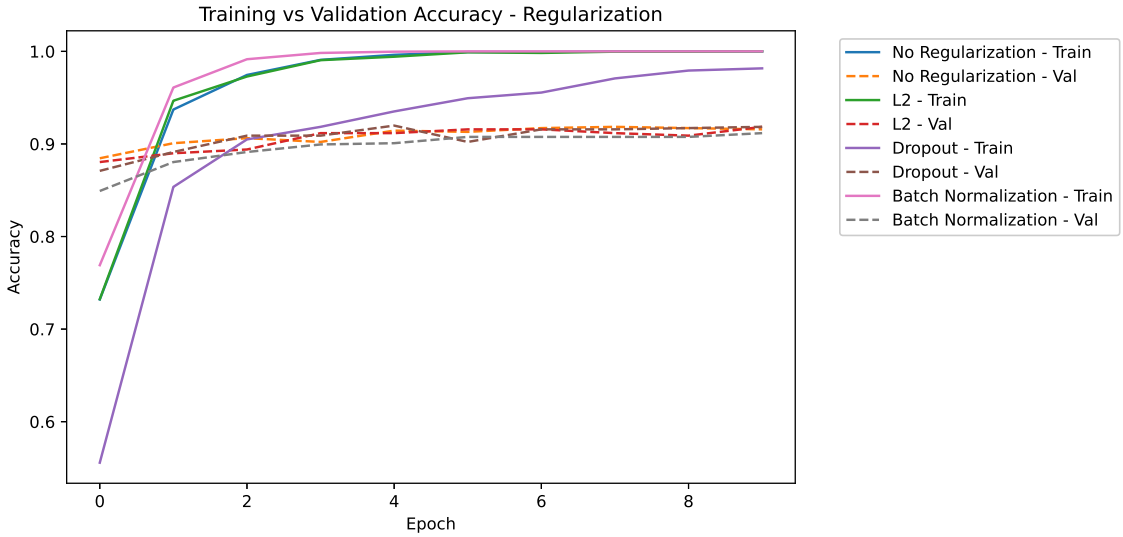


Figure 3: Training and Validation Accuracy for Different Regularization Methods

Training accuracy generally increases with epochs, while validation accuracy improves and may stabilize at a lower level,indicating a generalization gap. Regularization helps reduce overfitting by preventing the model from relying on specific training samples.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

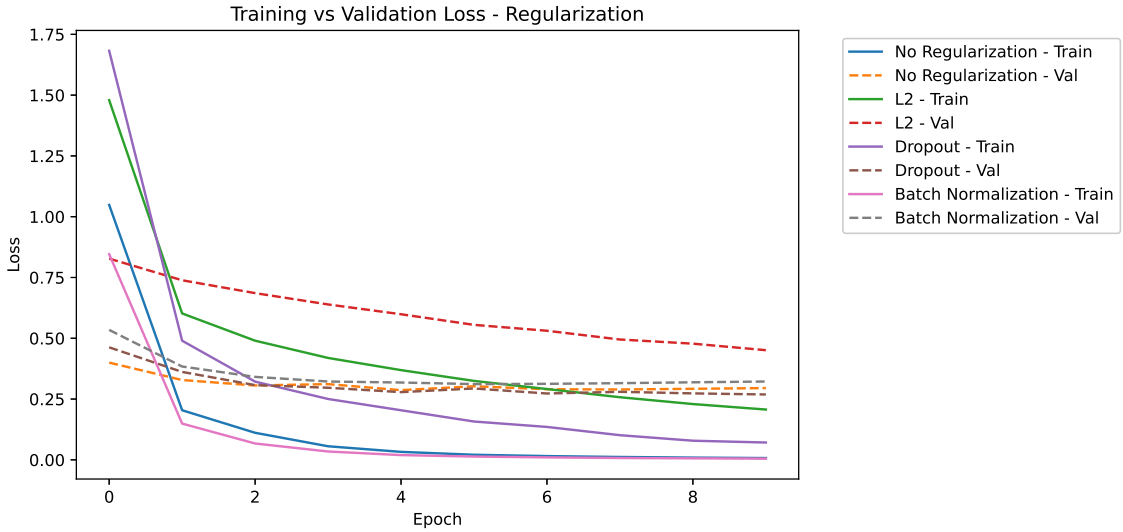


Figure 4: Training and Validation Loss for Different Regularization Methods

Training loss generally decreases as learning progresses,while validation loss may decrease initially and then stabilize or increase depending on the degree of overfitting.Regularization can reduce this divergence by improving the model's generalization to unseen data.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2+4+6+8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

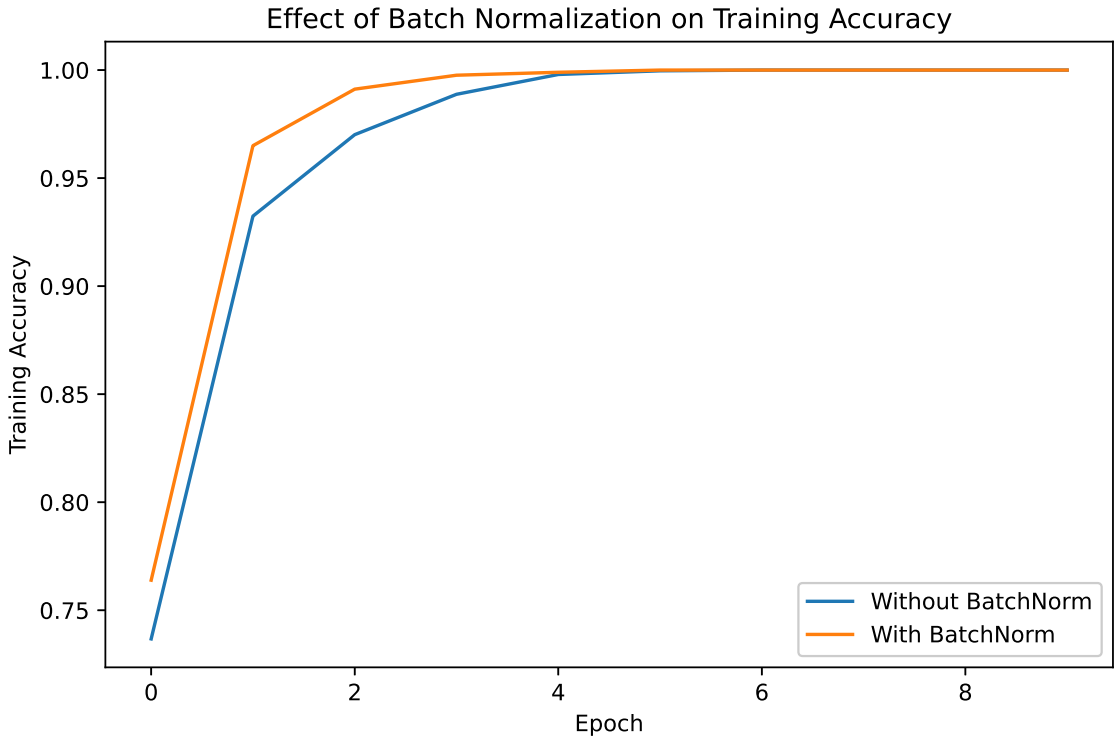


Figure 5: Validation Accuracy With and Without Batch Normalization

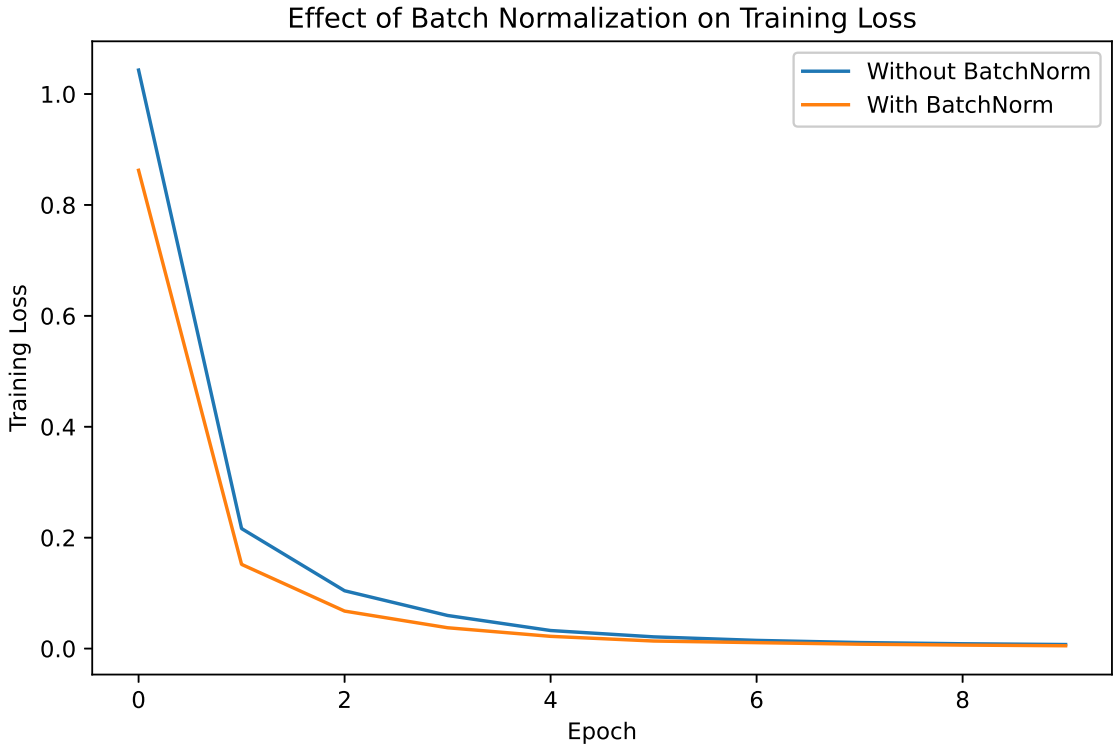


Figure 6: Validation Accuracy With and Without Batch Normalization

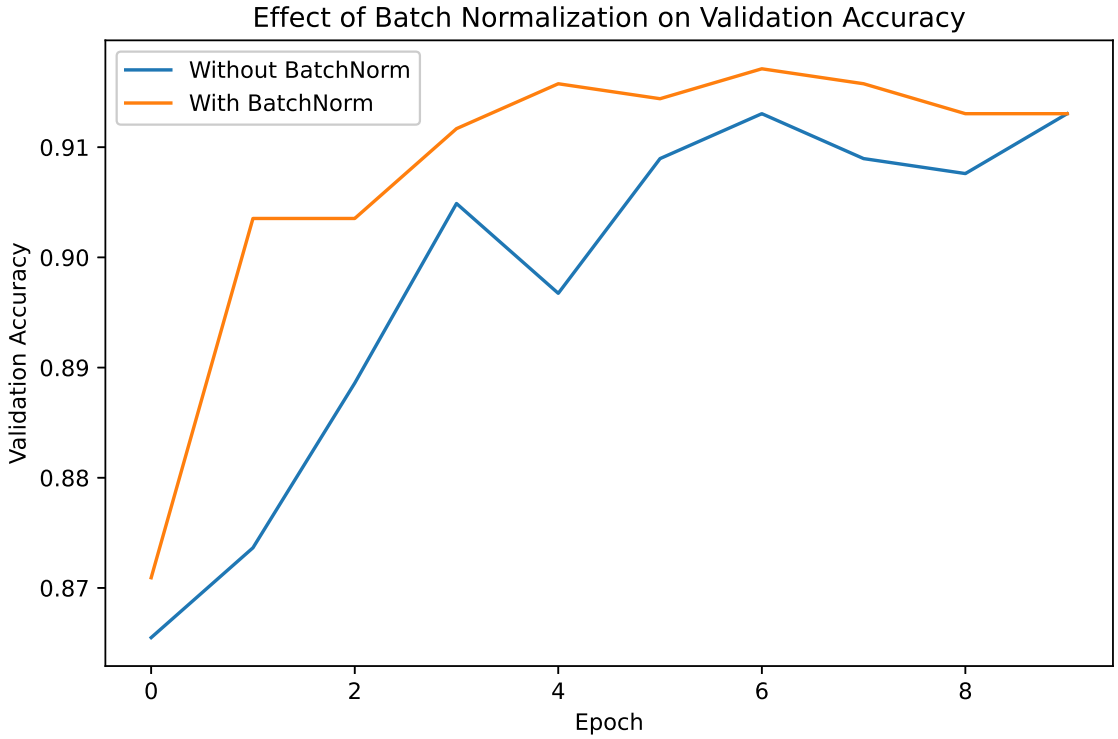


Figure 7: Validation Accuracy With and Without Batch Normalization

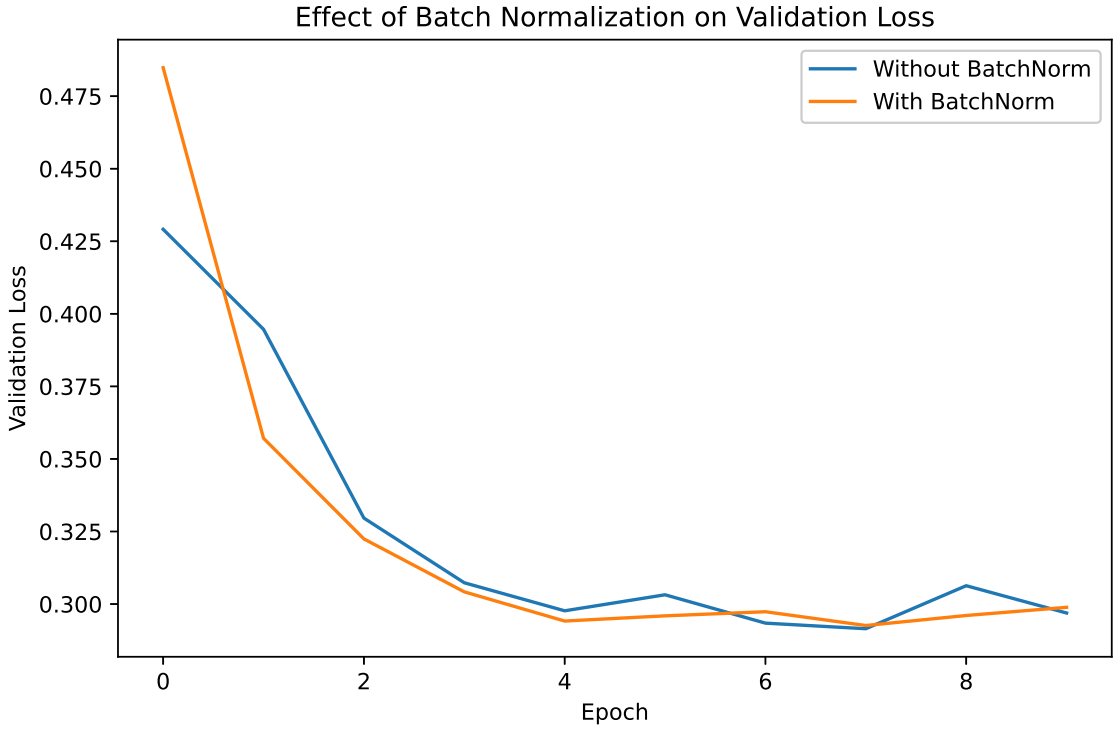


Figure 8: Validation Accuracy With and Without Batch Normalization

The plot compares validation accuracy when Batch Normalization is used and when it is not.The model with Batch Normalization shows more stable learning and may achieve better validation performance. This occurs because Batch Normalization normalizes intermediate activations which makes optimization more stable.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

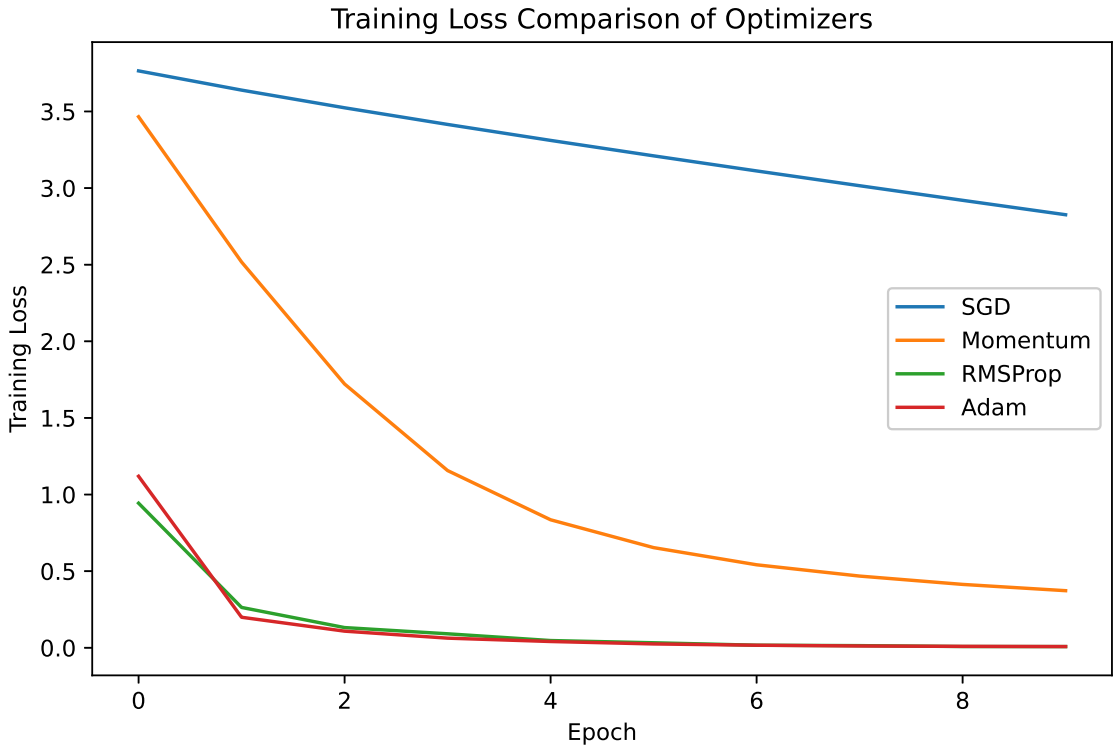


Figure 9: Training Loss vs. Epoch for Different Optimizers

The optimizers show different convergence rates and stability and some are reaching lower loss more quickly. These differences arise from the way each optimizer updates weights using gradient information and adaptive learning mechanisms.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

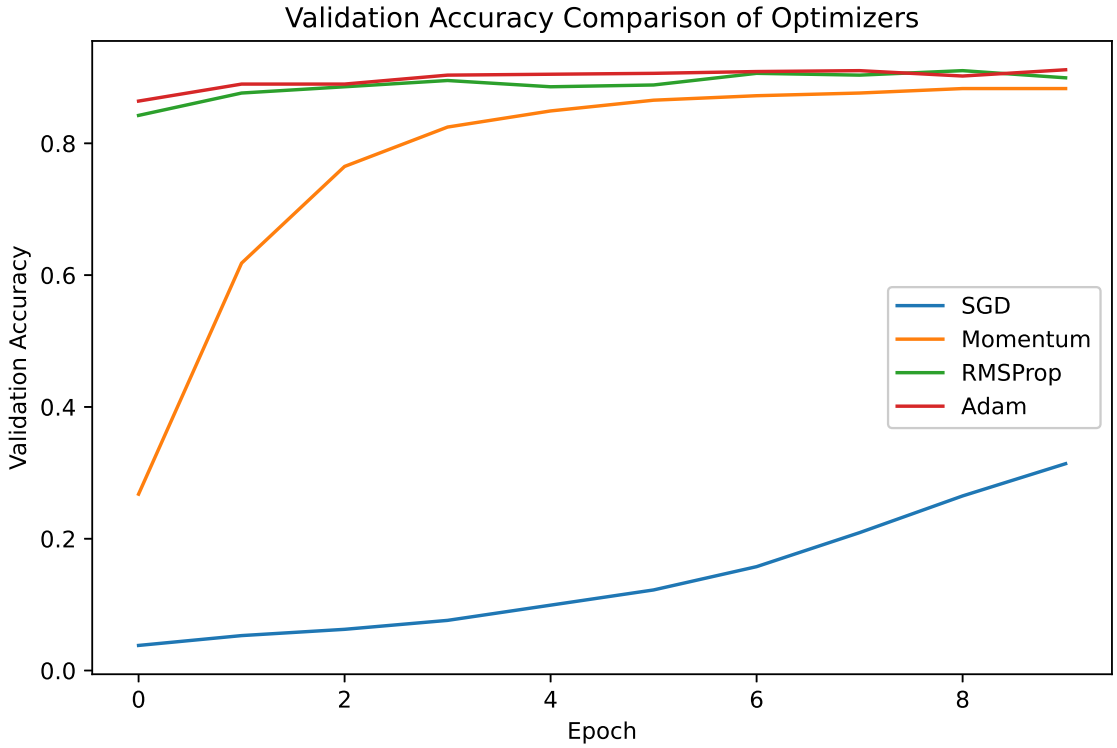


Figure 10: Validation Accuracy vs. Epoch for Different Optimizers

Table 1: Comparison of Different Optimizers

Optimizer	**Final Loss**	**Best Val. Accuracy**	**Epoch to Converge**	**Time (s)**
SGD	3.102	20.00%	9	2.5
Momentum	0.561	79.67%	10	1.8
RMSProp	0.002	87.67%	9	2.1
Adam	0.013	89.00%	7	2.1

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

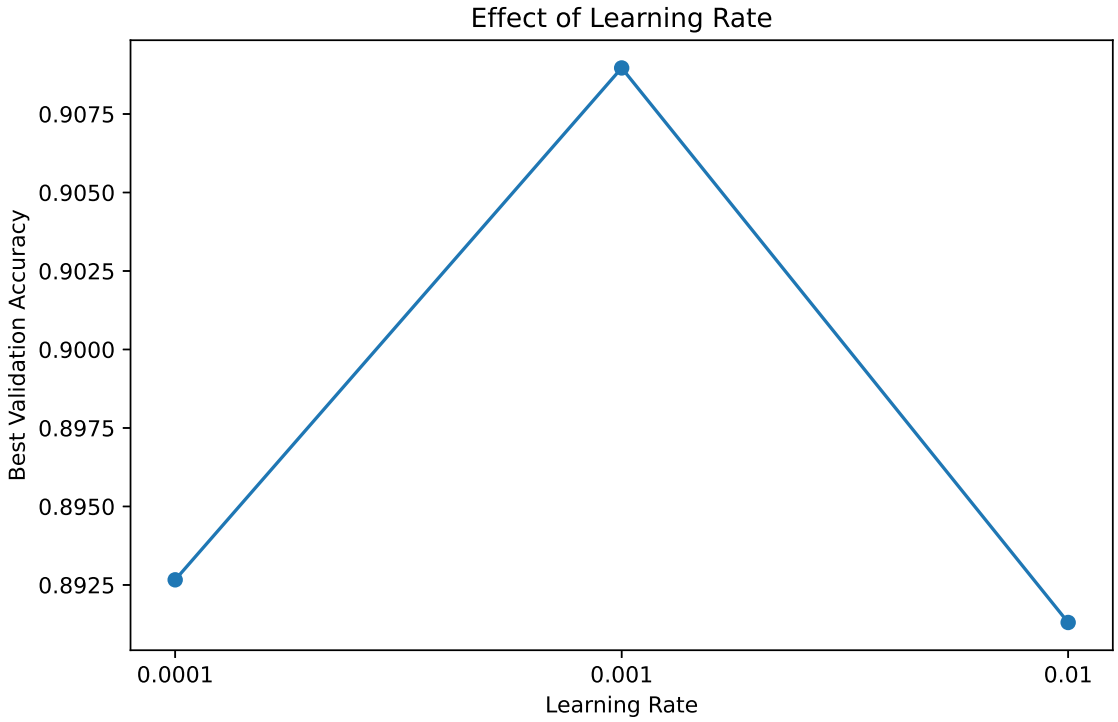


Figure 11: Effect of Learning Rate on Validation Accuracy

The plot indicate that validation performance depends strongly on the learning rate. A learning rate that is too large can cause unstable updates and one that is too small can slow convergence.

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

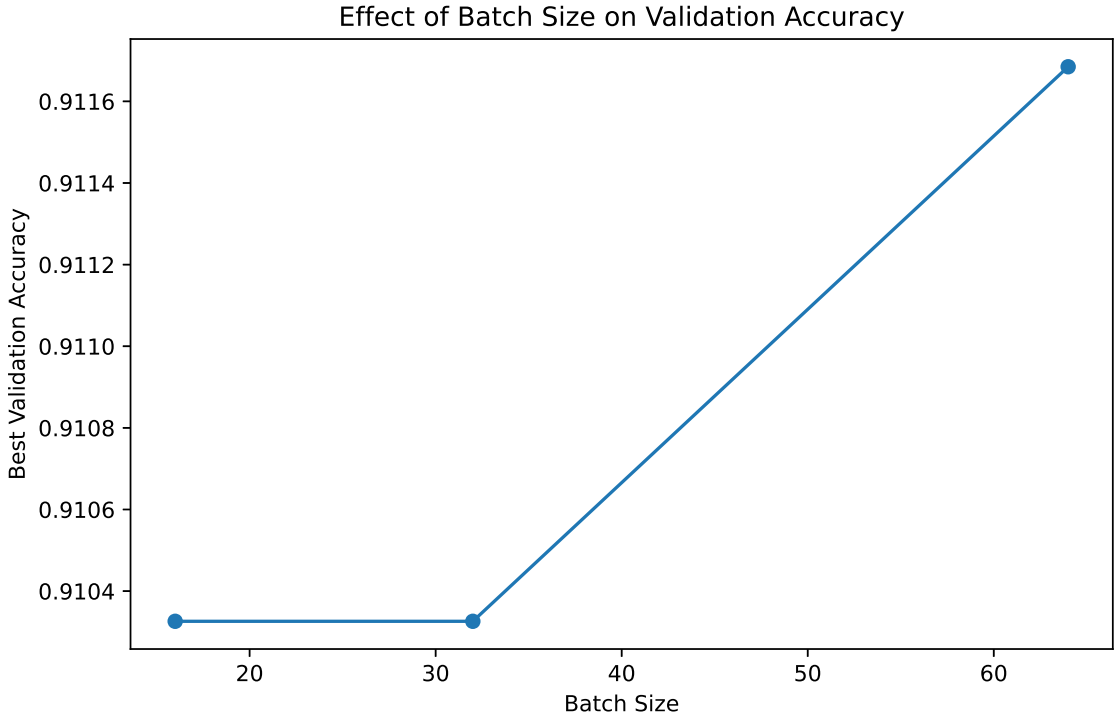


Figure 12: Effect of Batch Size on Validation Accuracy

The validation performance varies with batch size,so changing the number of samples processed in each update can affect generalization. Larger batches provide smoother gradient estimates and smaller batches introduce more variation in updates.

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

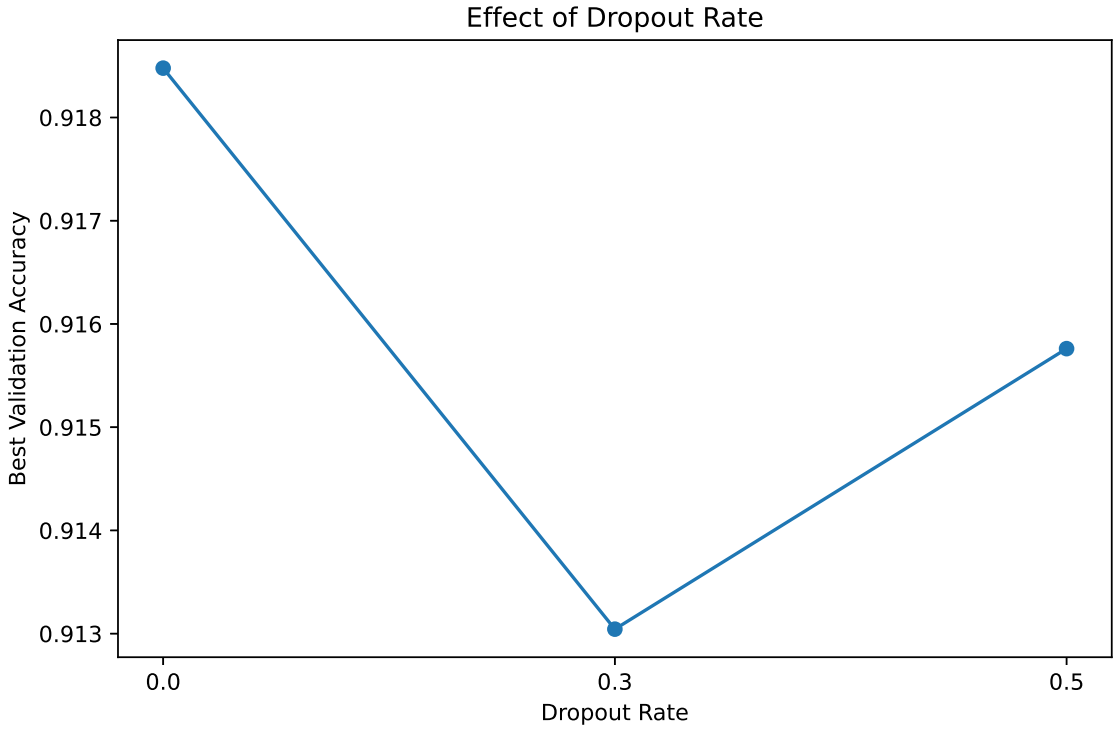


Figure 13: Effect of Dropout Rate on Validation Accuracy

Moderate dropout can improve validation performance by reducing overfitting and excessive dropout may reduce the model's ability to learn useful features. This occurs because dropout randomly deactivates neurons during training and therefore acts as regularization.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

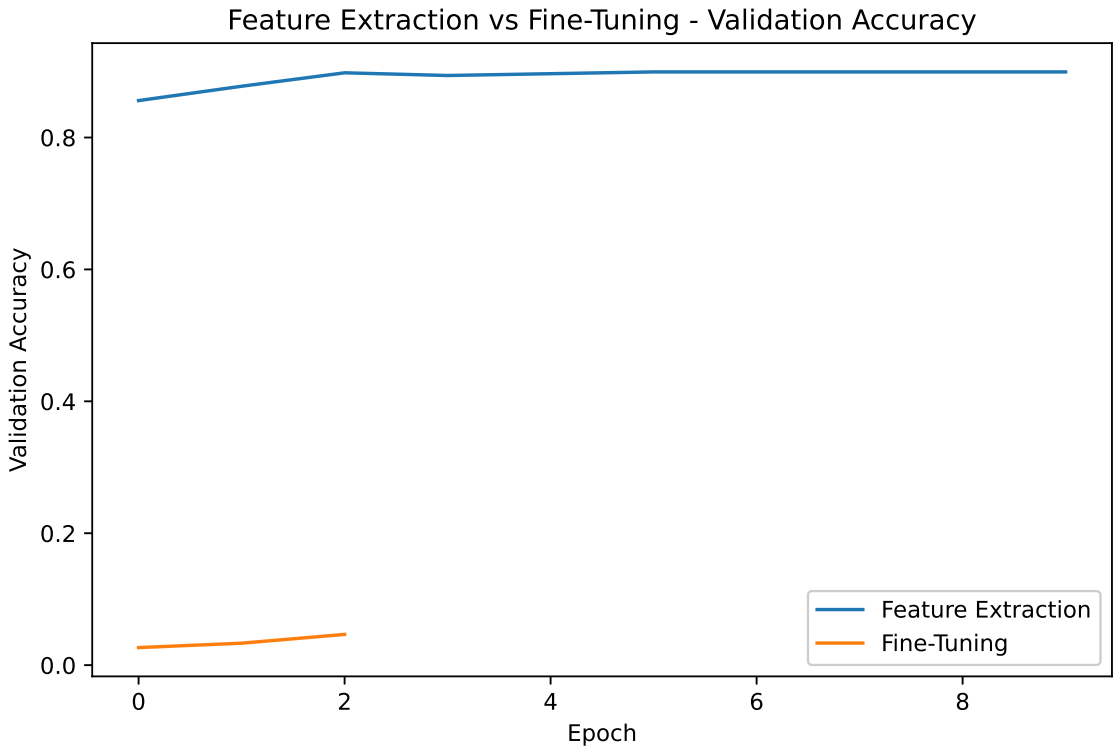


Figure 14: Validation Accuracy Comparison Between Feature Extraction and Fine-Tuning

Fine-tuning can improve validation performance by allowing selected pretrained layers to adapt to the target dataset. Feature extraction keeps the pretrained base fixed and while fine-tuning updates selected layers using a smaller learning rate.

Plot 12: Training and Validation Loss

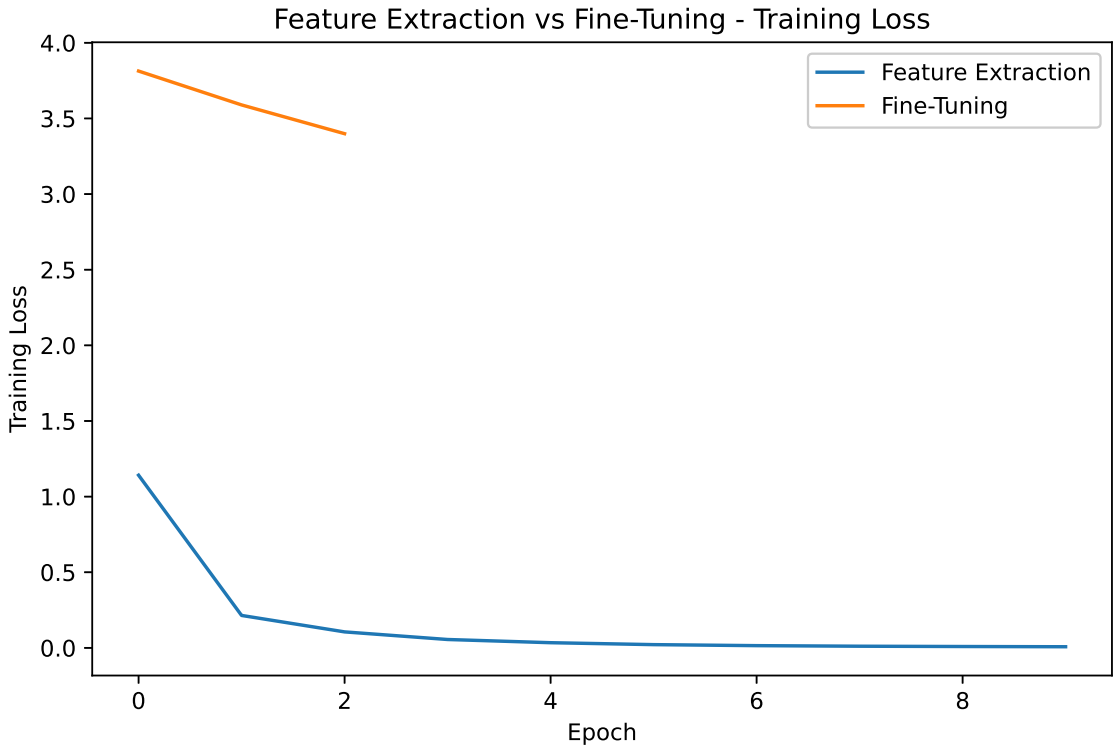


Figure 15: Training and Validation Loss Before and After Fine-Tuning

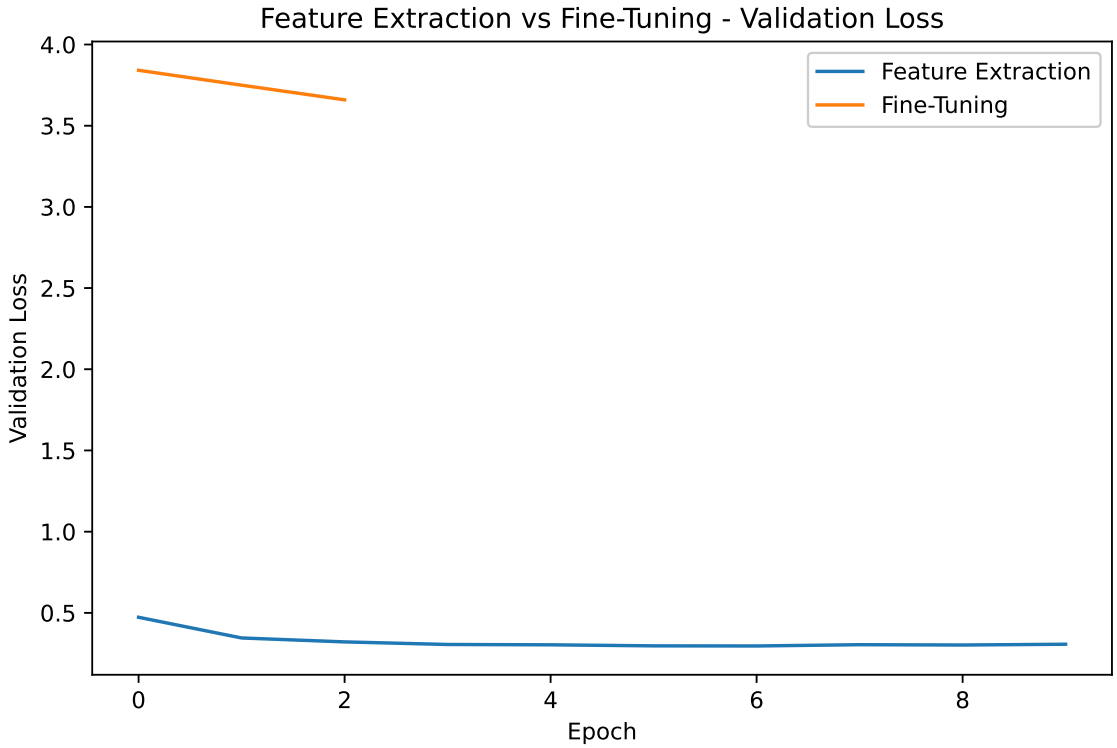


Figure 16: Training and Validation Loss Before and After Fine-Tuning

Fine-tuning can reduce the loss and improve adaptation of the pretrained network to the target dataset.A smaller learning rate is used to avoid making large changes to useful pretrained representations

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

- Fine-tuning normally requires a smaller learning rate because the pretrained network already contains useful features. Using a large learning rate may cause large weight updates that can destroy these learned features.A smaller learning rate allows the pretrained weights to be ajdusted gradually so that the network adapts to the target dataset while preserving the useful knowledge learned during pretraining.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i$$

and

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

Table 2: Five-Fold Cross-Validation Results

Configuration	**F1**	**F2**	**F3**	**F4**	**F5**	**Mean ± SD**
Best Initialization	0.8913	0.9032	0.9304	0.9236	0.8963	0.9090 ± 0.0172
Best Regularization	0.9100	0.8981	0.9253	0.9134	0.9065	0.9107 ± 0.0100
Best Hyperparameters	0.8947	0.9015	0.9321	0.9219	0.9099	0.9120 ± 0.0151
Alternative Optimizer	0.9100	0.8947	0.9372	0.9338	0.9116	0.9175 ± 0.0178

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

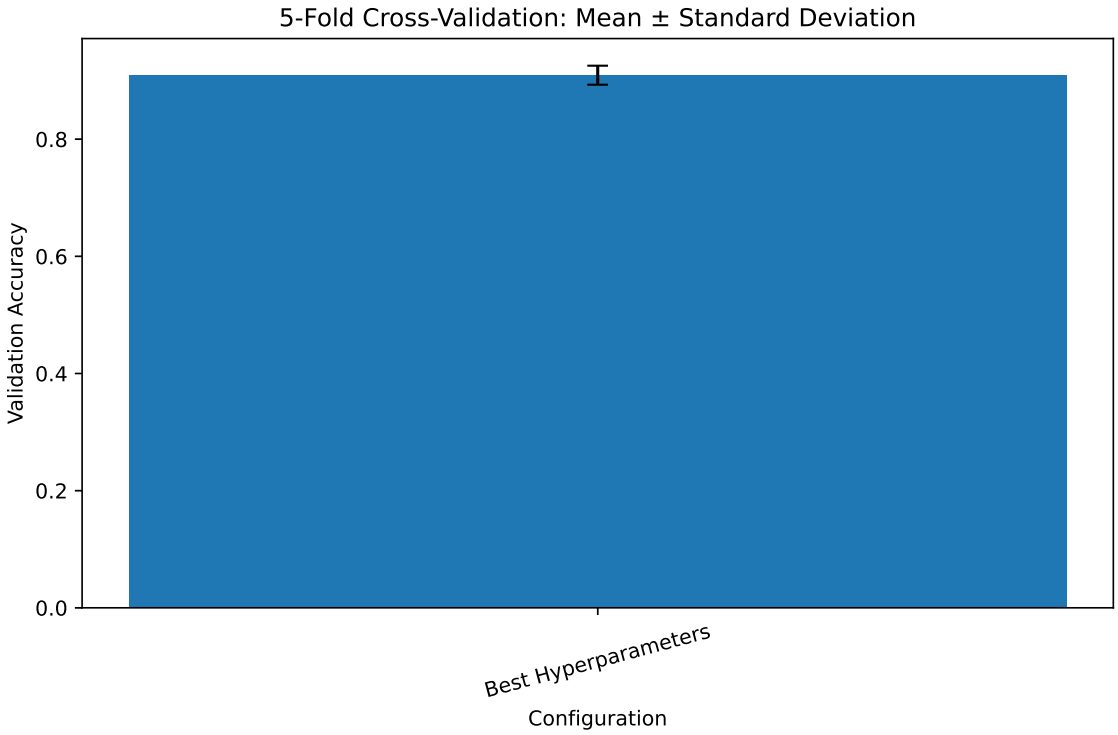


Figure 17: Mean Five-Fold Cross-Validation Accuracy with Standard Deviation

A higher mean accuracy indicates better average performance and smaller standard deviation indicates more consistent performance across folds. So both accuracy and variability should be considered when selecting the final configuration.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Table 3: Final Model Evaluation

Metric	**Value**
Mean CV Accuracy	91.75%
CV Standard Deviation	1.78%
Test Accuracy	89.92%
Precision	90.16%
Recall	89.92%
F1-score	89.88%
Training Time	5.28 seconds
Number of Parameters	337,445

Plot 14: Confusion Matrix

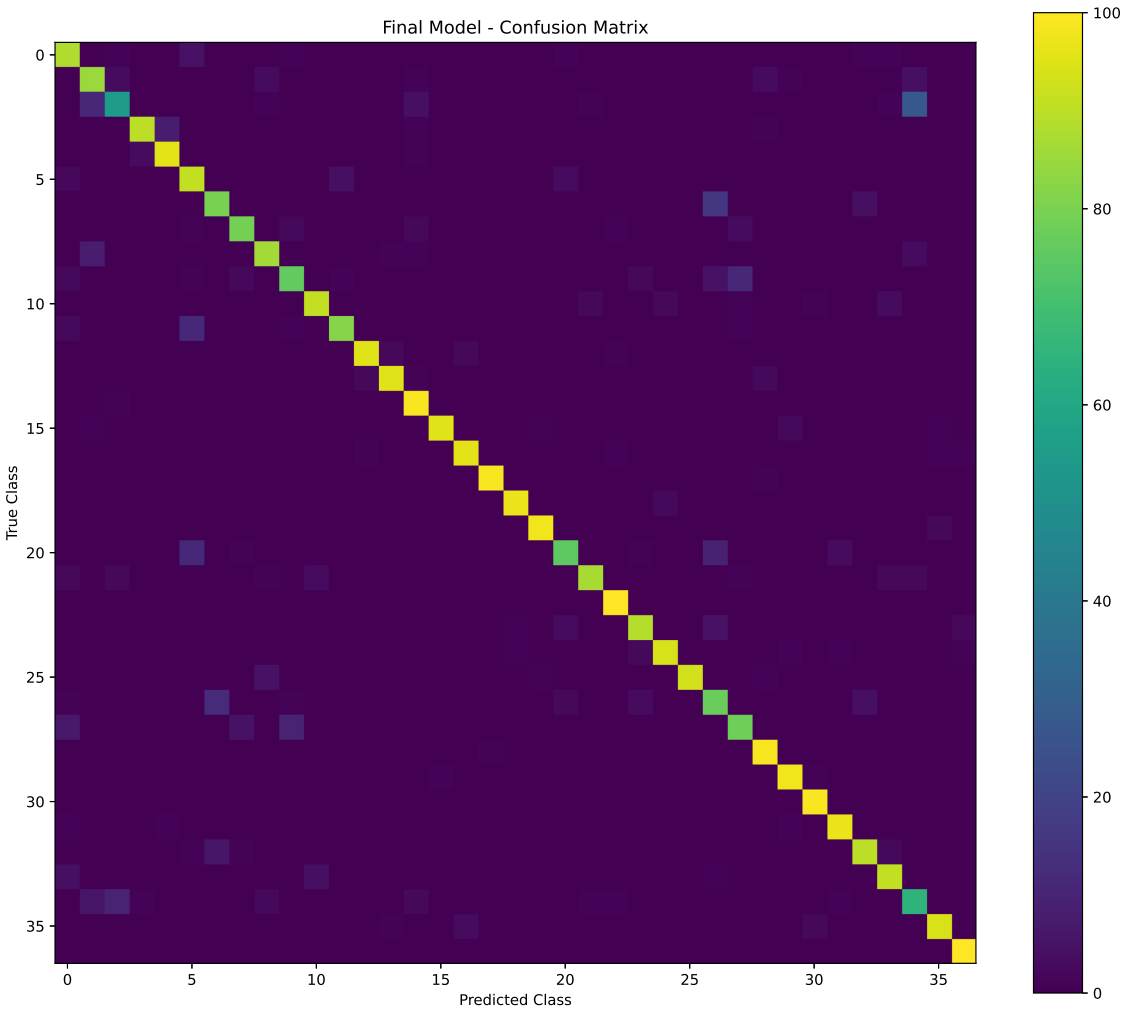


Figure 18: Confusion Matrix of the Final Model on the Independent Test Set

Confusion Matrix Inference:

The confusion matrix shows that most predictions lie along the main diagonal, indicating that the final model correctly classifies a large proportion of the test samples.The best classified classes include *Newfoundland*, *Yorkshire Terrier*, *German Shorthaired*, *Japanese Chin*, *Saint Bernard* and *Scottish Terrier*, which show strong diagonal responses.

The most frequent confusion is observed between *American Pit Bull Terrier* and *Staffordshire Bull Terrier*, with another noticeable confusion between *Birman* and *Ragdoll*.These errors may be caused by strong visual similarities such as shape, coat colour, fur texture, facial characteristics and overal appearance, making visually similar breeds difficult to distinguish.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	91.55%	0.25%	89.94%	27.25 s
Best Initialization	90.90%	1.57%	90.32%	7.04 s
Best Regularization	90.93%	1.65%	89.59%	5.57 s
Best Optimizer	91.30%	1.57%	89.70%	5.24 s
Best Hyperparameters	90.83%	1.52%	89.67%	7.28 s
Fine-Tuned Model	5.49%	2.88%	7.20%	209.52 s

Final Model Justification: The Baseline model was selected as the final model because it achieved highest CV accuracy of 91.55% with a low standard deviation of 0.25%.It required only 27.25 s of training time and achieved 89.94% test accuracy whihc indicating good generalization. Hence, it provides the best balance of accuracy, stability, computational cost and generalization.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

15. Discussion Questions

1. **What is the difference between model parameters and hyperparameters?**
   Model parameters are values learned automatically by model during training such as weights and biases. Hyperparameters are values set before training such as learning rate, batch size, dropout rate and optimizer.

2. **Why is weight initialization important?**
   Weight initialization determines the initial values of trainable weights before training begins.A good initialization help the model converge faster and reduce problems such as vanishing or exploding activations.

3. **Why can zero initialization be problematic for neural networks?**
   If all weights in a layer are initialized to zero, the neurons receive the same gradients and learn the same features.This creates a symmetry problem and prevents neurons from learning different representations. So random initialization is preferred for neural network weights.

4. **Compare Xavier and He initialization.**
   Xavier (Glorot) initialization is designed to maintain a suitable variance of activations and is commonly used with sigmoid or tanh activations.He initialization is designed mainly for ReLU-based networks and accounts for the fact that ReLU sets negative activations to zero.Since our classifeir uses ReLU, He initialization is generally more suitable.

5. **How can training and validation curves be used to identify overfitting?**
   Overfitting can be identified when training performance continues to improve while validation performance stops improving or becomes worse. If training loss continues to decrese while validation loss starts increasing, the model is likely memorizing the training data instead of generalizing well to unseen data.
6. **How does Dropout reduce overfitting?**
   Dropout randomly deactivates a fraction of neurons during training. This prevents the network from relying too heavily on particular neurons and makes it to learn more robust features.SO Dropout acts as a regularization technique and can improve generalization.
7. **What is the purpose of Batch Normalization?**
   Batch Normalization normalizes activations within a mini-batch during training. It helps make the optimization process more stable and can improve convergence.
8. **Explain the numerical Batch Normalization example.**
   For the batch $x = [2, 4, 6, 8]$, the mean is $\mu_B = 5$ and the variance is $\sigma_B^2 = 5$.The standard deviation is approximately $\sqrt{5} = 2.236$.Each value is normalized using

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

   The normalized values are then scaled and shifted using γ and β.

9. **What are the roles of γ and β?**
   γ and β are learnable parameters used after normalization. γ controls the scale of the normalized activation and β controls its shift.The final Batch Normalization output is given by

$$y_i = \gamma \hat{x}_i + \beta$$

10. **Compare SGD, Momentum, RMSProp and Adam.**
    SGD updates the weights directly using the current gradient.Momentum uses previous updates to accelerate movement in consistent directions and reduce oscillations.RMSProp adapts the learning rate using a moving average of

squared gradients.Adam combines momentum-like first-moment information with RMSProp-like second-moment adaptation.

11. **What happens when the learning rate is too large?**
A very large learning rate produces excessively large weight updates. The model may overshoot the minimum,causing the loss to oscillate or even diverge.So the training process can become unstable and fail to converge properly.

12. **What happens when the learning rate is too small?**
A very small learning rate produces very small weight updates.Training then becomes slow and may require many epochs to reach a good solution. The model can also appear to become stuck before reaching an optimal region.

13. **What is the effect of increasing batch size?**
Increasing the batch size means that more training samples are used to calculate each gradient update.This generally produces smoother and more stable gradient estimates,but it can require more memory and may reduce the beneficial noise present in smaller-batch training. In this experiment,batch sizes such as 16, 32 and 64 are compared.

14. **Explain stride and padding.**
Stride specifies how many pixels the kernel moves at each step.A larger stride reduces the spatial dimensions of the output. Padding adds extra pixels,usually zeros, around the input so that spatial information near the boundaries is preserved.
The convolution output size is
$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$
where N is the input size, P is padding, K is kernel size, and S is stride.

15. **Why is MobileNetV2 computationally efficient?**
MobileNetV2 is designed as a lightweight CNN that reduces computational complexity while maintaining useful image representations.It uses depthwise convolutions, pointwise 1×1 convolutions, inverted residual blocks and linear bottlenecks.This makes it sutiable for CPU-based laboratory experiments.

16. **What is depthwise separable convolution?**
Depthwise separable convolution separates a standard convolution into two operations: depthwise convolution and pointwise 1×1 convolution.Depthwise convolution applies a spatial filter separately to each input channel, while pointwise convolution combines the resulting channels.This greatly reduces the number of computations and parameters compared with a standard convolution.

17. **What is transfer learning?**
Transfer learning uses knowledge learned by a model on a large dataset and applies it to a new related task.In this experiment, MobileNetV2 pretrained on ImageNet was used as the pretrained base. Instead of learning all visual features from scratch, the pretrained features were adapted to our target classification problem.

18. **Differentiate feature extraction and fine-tuning.**
In feature extraction, the pretrained MobileNetV2 base is frozen and only the newly added classifier layers are trained.In fine-tuning, selected layers of the pretrained base are unfrozen and updated along with the classifier.Feature extraction is faster, while fine-tuning allows the pretrained features to adapt more specifically to the target dataset.

19. **Why is a smaller learning rate generally used during fine-tuning?**
The pretrained network already contains useful features learned from ImageNet.A large learning rate could change these weights too aggressively destroying useful pretrained representations.So a smaller learning rate is generally used so that the pretrained features are adjusted gradually to the target dataset.

20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**
K-Fold Cross-Validation divides the training data into K folds and uses different folds for validation across multiple runs.This gives a more reliable estimate of how a configuration performs on unseen data and reduces dependence on a single train-validation split.

21. **Why must the test set remain untouched during tuning?**
The test set is intended to provide an unbiased estimate of the final model's generalization performance.If it is repeatedly used during hyperparameter tuning, information from the test set can influence model selection and cause data leakage.

22. **Why should mean and standard deviation both be reported?**

The mean validation accuracy represents the average performance of a configuration across the folds. The standard deviation represents how much the performance varies between folds. Therefore, reporting both mean ± SD gives information about both performance and stability.

23. **Is the highest validation accuracy always sufficient to select a model?**
No.The highest validation accuracy alone is not always sufficient. Model selection should also consider cross-validation variability, generalization, computational cost and stability. Five-fold cross-validation was used to compare configurations using both mean accuracy and standard deviation before selecting the final configuration.

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

Table 4: Additional Configurations Evaluated Using 5-Fold Cross-Validation

Configuration	**Learning Rate**	**Dropout**	**Batch Size**	**Optimizer**	**Mean ± SD**
Additional Configuration A	0.0001	0.25	64	Adam	0.8991 ± 0.0108
Additional Configuration B	0.001	0.50	64	Adam	0.9212 ± 0.0136

Table 5: 5-Fold Cross-Validation Results

Configuration	**F1**	**F2**	**F3**	**F4**	**F5**	**Mean ± SD**
Additional Configuration A	0.8862	0.8930	0.9151	0.9083	0.8929	0.8991 ± 0.0108
Additional Configuration B	0.9049	0.9083	0.9406	0.9321	0.9201	0.9212 ± 0.0136

Inference and Conclusion: Additional Configuration B achieved a higher mean cross-validation accuracy of 92.12% compared with 91.75% for the selected configuration, indicating better average validation performance. It also obtained a lower standard deviation of 1.36% compared with 1.78%, indicating more consistent performance across the five folds. Both configurations use feature extraction without fine-tuning and therefore have relatively low computational cost compared with fine-tuning. However, the additional configuration was not evaluated on the independent test set, whereas the selected configuration achieved a final test accuracy of 89.92%. Therefore, although Additional Configuration B is promising based on CV accuracy, stability, and computational cost, it cannot be concluded to be preferable based on final test performance.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`